May 21, 2025 at 8:27 AM

You said:

list of all C# data structure like— string, date, list, dictionary, set, stack, queue, praiorityQueue (min heap, max heap), linked list, tree, try, tuple, record. with example and all usefull function associated with them are need in problem solving. as well ass list all i missed . dont miss anything that we get after clicking dot after the data structure

Memory updated

ChatGPT said:

Here is a comprehensive guide to all core **C# data structures**, including examples and **commonly used members** (properties, methods, and operators)—ideal for **problem-solving, interviews, and advanced development**:

✅ 1. `string`

Immutable sequence of characters.

C#

```
string name = "Afifa";
```

Useful members:

C#

```
name.Length
name.ToUpper(), name.ToLower()
name.Contains("fi")
name.StartsWith("A"), name.EndsWith("a")
name.IndexOf("i"), name.LastIndexOf("f")
name.Substring(1, 2)
name.Replace("fi", "xy")
name.Split(' ')
```

```
string.Join(",", array)
string.IsNullOrEmpty(name), string.IsNullOrWhiteSpace(name)
```

✅ 2. `DateTime`

Used for date and time manipulation.

```
C#

DateTime now = DateTime.Now;
DateTime specific = new DateTime(2024, 12, 25);
```

Useful members:

```
C#

now.Year, now.Month, now.Day
now.AddDays(5), now.AddMonths(-1)
now.Subtract(anotherDate)
now.ToString("yyyy-MM-dd")
DateTime.TryParse("2024-05-01", out var dt)
```

✅ 3. `List<T>`

Dynamic array.

```
C#

List<int> numbers = new() {1, 2, 3};
```

Useful members:

```
C#

numbers.Add(4), numbers.AddRange(new[] {5, 6})
numbers.Remove(3), numbers.RemoveAt(0)
numbers.Insert(1, 99)
numbers.Contains(2)
numbers.IndexOf(2)
numbers.Sort(), numbers.Reverse()
numbers.Count, numbers.Clear()
numbers.ForEach(Console.WriteLine)
```

✅ 4. `Dictionary<TKey, TValue>`

Key-value pairs.

```
C#

var dict = new Dictionary<string, int> { ["apple"] = 3 };
```

Useful members:

```
C#

dict.Add("banana", 5)
dict.Remove("apple")
dict.ContainsKey("banana")
dict.ContainsValue(5)
dict["banana"] = 6
dict.Keys, dict.Values
dict.TryGetValue("banana", out var value)
```

✅ 5. `HashSet<T>`

Unordered collection with unique elements.

```
C#

HashSet<int> set = new() {1, 2, 3};
```

Useful members:

```
C#

set.Add(4), set.Remove(2)
set.Contains(3)
set.UnionWith(otherSet), set.IntersectWith()
set.ExceptWith(), set.SymmetricExceptWith()
```

✅ 6. `Stack<T>`

LIFO data structure.

C#

```
Stack<int> stack = new();
stack.Push(1);
int top = stack.Pop();
```

Useful members:

C#

```
stack.Push(5), stack.Pop()
stack.Peek(), stack.Contains(3)
stack.Count, stack.Clear()
```

✅ 7. `Queue<T>`

FIFO data structure.

C#

```
Queue<string> queue = new();
queue.Enqueue("task1");
string next = queue.Dequeue();
```

Useful members:

C#

```
queue.Enqueue(), queue.Dequeue()
queue.Peek(), queue.Contains()
queue.Count, queue.Clear()
```

✅ 8. `PriorityQueue<TElement, TPriority>` (Min-Heap by default)

C#

```
var pq = new PriorityQueue<string, int>();
pq.Enqueue("task1", 1);
pq.Enqueue("task2", 0);
```

Useful members:

C#

```
pq.Enqueue("item", priority)
pq.Dequeue(), pq.Peek()
pq.Count, pq.Clear()
```

➡️ **Max-Heap** workaround:

C#

```
pq.Enqueue("item", -priority)  // invert the priority
```

✅ 9. `LinkedList<T>`

C#

```
LinkedList<int> list = new();
list.AddLast(1); list.AddFirst(0);
```

Useful members:

C#

```
list.AddFirst(value), list.AddLast(value)
list.AddBefore(node, value), list.AddAfter(node, value)
list.RemoveFirst(), list.RemoveLast()
list.Find(value), list.Contains(value)
```

✅ 10. `Tuple<T1, T2, ..., T7>`

C#

```
var t = Tuple.Create("Afifa", 100);
var name = t.Item1;
```

For better readability use `ValueTuple`:

C#

```
(string name, int score) = ("Afifa", 100);
```

✅ 11. `Record` (C# 9+)

Immutable reference type with value-based equality.

```
C#
public record Person(string Name, int Age);
var p1 = new Person("Afifa", 25);
```

Benefits:

- Immutable
- Structural equality
- Deconstruction

✅ 12. `Tree` (Custom)

```
C#
class TreeNode {
    public int Value;
    public List<TreeNode> Children = new();
}
```

Useful for hierarchical data like file systems, org charts, etc.

✅ 13. `Trie` (Custom)

Prefix tree used in string searching/autocomplete.

```
C#
class TrieNode {
    public Dictionary<char, TrieNode> Children = new();
    public bool IsWord = false;
}
```

✅ 14. `Array`

C#

```
int[] arr = {1, 2, 3};
```

Useful members:

C#

```
arr.Length
Array.Sort(arr), Array.Reverse(arr)
Array.IndexOf(arr, 2)
Array.Copy(source, dest, length)
```

✅ 15. `Span<T>` / `ReadOnlySpan<T>`

Efficient memory slicing (C# 7.2+).

C#

```
Span<int> span = stackalloc int[3];
```

Use in performance-critical scenarios.

✅ 16. `IEnumerable<T>` / `IQueryable<T>`

Used for LINQ queries and deferred execution.

C#

```
var filtered = numbers.Where(x => x > 2);
```

✅ 17. `ObservableCollection<T>`

Collection with change notifications (used in WPF).

C#

```
ObservableCollection<string> names = new();
names.CollectionChanged += ...
```

✅ 18. `SortedList<TKey, TValue>` / `SortedDictionary<TKey, TValue>`

Keeps keys in sorted order.

```
C#

SortedList<string, int> sorted = new();
```

✅ 19. `ConcurrentDictionary`, `ConcurrentQueue`, etc.

Thread-safe collections.

```
C#

ConcurrentDictionary<string, int> cache = new();
```

✅ 20. `ImmutableList<T>`, `ImmutableDictionary<TKey, TValue>`

Immutable versions for functional-style programming.

✅ 21. `BitArray`

Efficient binary flags.

```
C#

BitArray flags = new(10); flags[0] = true;
```

🔍 IntelliSense Tips

For any structure:

```
C#

var s = new Stack<int>();
s.  // Press dot to see all members like Push, Pop, Count
```

Use `Ctrl+.` in Visual Studio or type `.` to explore all **properties, methods, extension methods**.

🔚 Summary Table

Data Structure	Namespace	Purpose
`string`	`System`	Text
`DateTime`	`System`	Time
`List<T>`	`System.Collections.Generic`	Dynamic array
`Dictionary<K,V>`	`System.Collections.Generic`	Key-value
`HashSet<T>`	`System.Collections.Generic`	Unique set
`Stack<T>`	`System.Collections.Generic`	LIFO
`Queue<T>`	`System.Collections.Generic`	FIFO
`PriorityQueue<TElement, TPriority>`	`System.Collections.Generic`	Min-Heap
`LinkedList<T>`	`System.Collections.Generic`	Bi-directional nodes
`Tuple`, `ValueTuple`	`System`	Group values
`Record`	`System`	Immutable type
`Array`, `Span<T>`	`System`	Performance
`Trie`, `Tree`	Custom	Hierarchical/string search
`Concurrent*`	`System.Collections.Concurrent`	Thread-safe
`Immutable*`	`System.Collections.Immutable`	Immutable variants

Data Structure	Namespace	Purpose
`BitArray`	`System.Collections`	Bits
`SortedList`, `SortedDictionary`	`System.Collections.Generic`	Sorted

Would you like a downloadable PDF or code snippets in a GitHub Gist for this?

You said:

rewrite all with example, you listout but missed give example as well as you missed many thing like array.sort() or revers() with param, max heap (a,b) = > (a-b) like function... array.fill(with param) linked list, double linked list etc

ChatGPT said:

You're right—let's now provide **complete examples with code, IntelliSense-accessible members**, and **problem-solving tips** for each C# data structure. I'll also cover **missing** structures like `Array.Sort(…)`, `Array.Fill(…)`, `SortedSet`, `Deque`, `MaxHeap`, etc.

🔹 1. `string`

```C#
string name = "Afifa";
Console.WriteLine(name.ToUpper());        // AFIFA
Console.WriteLine(name.Contains("fi"));   // True
Console.WriteLine(name.Substring(1, 2));  // fi
```

Useful members:

```C#
Length, Contains, Substring, IndexOf, Replace, Split, StartsWith, EndsWith, ToUpper, ToLo
```

🔹 2. `DateTime`

```C#
DateTime now = DateTime.Now;
DateTime birthday = new DateTime(1993, 12, 25);
var tomorrow = now.AddDays(1);
Console.WriteLine(birthday.DayOfWeek);  // Saturday
```

Useful members:

```C#
Year, Month, Day, AddDays, AddMonths, Subtract, ToString("yyyy-MM-dd"), TryParse
```

🔹 3. `Array`

```C#
int[] arr = { 5, 2, 3, 1 };

// Sort ascending
Array.Sort(arr);
Console.WriteLine(string.Join(", ", arr)); // 1, 2, 3, 5

// Sort descending
Array.Sort(arr, (a, b) => b - a);

// Fill array
Array.Fill(arr, 9); // All elements become 9

// Reverse
Array.Reverse(arr);
```

🔹 4. `List<T>`

```C#
List<int> numbers = new() { 1, 3, 5 };
numbers.Add(7);
numbers.Insert(1, 2);  // [1, 2, 3, 5, 7]
```

```
numbers.Sort();        // Ascending
numbers.Sort((a, b) => b - a);  // Descending
```

Members:

```
C#

Add, AddRange, Remove, RemoveAt, Contains, IndexOf, Insert, Sort, Reverse, Clear, Count,
```

🔹 5. `Dictionary<TKey, TValue>`

```
C#

Dictionary<string, int> dict = new()
{
    ["apple"] = 3,
    ["banana"] = 2
};
dict["banana"] = 5;
if (dict.TryGetValue("banana", out var val)) Console.WriteLine(val);
```

🔹 6. `HashSet<T>` – Unordered, unique

```
C#

HashSet<int> set = new() { 1, 2, 3 };
set.Add(3);  // ignored
set.UnionWith(new[] { 4, 5 }); // adds 4 and 5
```

🔹 7. `SortedSet<T>` – Sorted, unique

```
C#

SortedSet<int> sorted = new() { 3, 1, 2 };
foreach (var i in sorted) Console.WriteLine(i); // 1 2 3
```

🔹 8. `Stack<T>` – LIFO

C#

```
Stack<int> stack = new();
stack.Push(10);
int last = stack.Pop();     // 10
```

🔹 9. `Queue<T>` – FIFO

C#

```
Queue<string> queue = new();
queue.Enqueue("first");
string item = queue.Dequeue();  // "first"
```

🔹 10. `PriorityQueue<TElement, TPriority>` – Min-Heap

C#

```
PriorityQueue<string, int> pq = new();
pq.Enqueue("low", 1);
pq.Enqueue("high", 0);
Console.WriteLine(pq.Dequeue()); // "high"
```

🔸 Max Heap:

C#

```
PriorityQueue<string, int> maxHeap = new(Comparer<int>.Create((a, b) => b - a));
maxHeap.Enqueue("high", 10);
maxHeap.Enqueue("low", 1);
```

🔹 11. `LinkedList<T>` – Double-linked list

C#

```
var list = new LinkedList<int>();
list.AddFirst(1);
list.AddLast(3);
var two = list.AddAfter(list.First, 2);  // 1 -> 2 -> 3
```

🔹 12. `Tuple<T1, T2>` and `ValueTuple`

```C#
var tuple = Tuple.Create("Afifa", 11);
var (name, age) = ("Afifa", 11); // ValueTuple
```

🔹 13. `Record` – Immutable value object (C# 9+)

```C#
public record Person(string Name, int Age);

var p1 = new Person("Afifa", 30);
var p2 = p1 with { Age = 31 };
```

🔹 14. `Tree` (Custom)

```C#
class TreeNode {
    public int Val;
    public List<TreeNode> Children = new();
}
```

🔹 15. `Trie` (Prefix tree for strings)

```C#
class TrieNode {
    public Dictionary<char, TrieNode> Children = new();
    public bool IsEndOfWord;
}
```

🔹 16. `Span<T>` – Efficient slicing (C# 7.2+)

C#

```
Span<int> s = stackalloc int[3];
s[0] = 1;
```

🔹 17. `ObservableCollection<T>` – CollectionChanged for WPF

C#

```
var collection = new ObservableCollection<string>();
collection.CollectionChanged += (s, e) => Console.WriteLine("Changed");
collection.Add("Afifa");
```

🔹 18. `SortedList<TKey, TValue>` vs `SortedDictionary<TKey, TValue>`

C#

```
SortedList<string, int> sortedList = new() { ["a"] = 1, ["b"] = 2 };
SortedDictionary<string, int> sortedDict = new() { ["a"] = 1, ["b"] = 2 };
```

🔹 19. `ConcurrentDictionary<TKey, TValue>` – Thread-safe

C#

```
ConcurrentDictionary<string, int> cd = new();
cd.TryAdd("x", 1);
```

🔹 20. `ImmutableList<T>` – Functional programming

C#

```
var list = ImmutableList.Create<int>(1, 2, 3);
var newList = list.Add(4); // returns new list
```

🔹 21. `BitArray` – Efficient flags

```
C#

BitArray flags = new(8);
flags[0] = true;
Console.WriteLine(flags[0]); // True
```

✅ Summary Table

Data Structure	Usage	Key Functions
`string`	Text	Substring, Replace, IndexOf
`DateTime`	Dates/times	AddDays, Subtract, ToString
`Array`	Static list	Sort, Reverse, Fill
`List<T>`	Dynamic list	Add, Insert, Sort, Remove
`Dictionary<K,V>`	Key-value pairs	TryGetValue, ContainsKey
`HashSet<T>`	Unique values	UnionWith, IntersectWith
`SortedSet<T>`	Sorted unique values	Min/Max operations
`Stack<T>`	LIFO	Push, Pop, Peek
`Queue<T>`	FIFO	Enqueue, Dequeue
`PriorityQueue<T>`	Min/Max heap	Enqueue, Dequeue (with comparer)
`LinkedList<T>`	Bidirectional list	AddBefore, AddAfter
`Tuple`, `ValueTuple`	Grouping values	Item1, Item2 or named deconstruction
`Record`	Immutable class	with, equality, ToString
`Tree`, `Trie`	Hierarchical / Prefix structures	Custom classes

Data Structure	Usage	Key Functions
`Span<T>`	Memory slice (perf)	Efficient iteration
`ObservableCollection<T>`	WPF UI updates	CollectionChanged
`SortedList`, `SortedDict`	Auto-sorted	Keys, Values
`Concurrent*`	Thread-safe collections	TryAdd, TryRemove
`Immutable*`	Functional style	Add, Remove (new copy)
`BitArray`	Memory-efficient flags	Set, Get

Would you like this as a **PDF**, **cheat sheet**, or **interactive notebook**? I can also help you prepare questions or practice problems based on these structures.

You said:

small = new PriorityQueue<int, int>
(Comparer<int>.Create((a, b) => b.CompareTo(a)));
is there any alternative

ChatGPT said:

Yes, the line:

```
C#
small = new PriorityQueue<int, int>(Comparer<int>.Create((a, b) => b.CompareTo(a)));
```

creates a **max-heap** using `PriorityQueue<TElement, TPriority>` by reversing the default comparer. This is **correct and idiomatic**, but there are a few **alternatives and variations**.

✅ Alternative 1: Use `Comparer<int>.Default.Reverse()`

This is more expressive and readable:

C#

```
small = new PriorityQueue<int, int>(Comparer<int>.Default.Reverse());
```

> `Reverse()` is available on `Comparer<T>` in .NET 6+.

✅ Alternative 2: Explicit `new Comparer<int>` with lambda

If you can't use `.Reverse()` (e.g., older C# version):

C#

```
small = new PriorityQueue<int, int>(
    Comparer<int>.Create((x, y) => y - x) // or y.CompareTo(x)
);
```

✅ Alternative 3: Use negative priorities

Another trick, especially in **competitive coding**:

C#

```
PriorityQueue<int, int> pq = new();
pq.Enqueue(10, -10); // store priority as -priority

var max = pq.Dequeue(); // element with highest original value
```

But be careful—this assumes:

- You control how priorities are added.
- All priorities are numeric and convertible to negative.

🔁 Summary

Approach	Code Example	Notes
Default + Reverse()	`Comparer<int>.Default.Reverse()`	✅ Clean, readable

Approach	Code Example	Notes
Custom lambda	`Comparer<int>.Create((a, b) => b - a)`	✅ Common trick
Use negative priorities	`Enqueue(value, -priority)`	⚠️ Manual, less safe

Let me know if you'd like to wrap this into a reusable helper or extension method for MaxHeap!

Today 1:54 PM

You said:

make mark down, avoid duplicate

ChatGPT said:

This response couldn’t load

You said:

make markdown file , avoid duplicate information If any

Analyzing